HIGH-LEVEL DESIGN (HLD)

RAG-BASED CUSTOMER SUPPORT ASSISTANT

(Retrieval-Augmented Generation with Workflow Routing & HITL)

Submitted by:
Sai Kadiravan S Intern ID : IN126008102

Institution:
Innomatics Research Labs

Project Type:
AI / Machine Learning Internship Project

Knowledge Base Used:
Customer Service Handbook (PDF containing service policies, complaint handling procedures, and customer interaction guidelines)

Technologies Used:
Python, LangChain, ChromaDB, Groq LLM, HuggingFace Embeddings

Submission Date:
23/04/2026

TABLE OF CONTENTS

1. System Overview
    1.1 Problem Definition
    1.2 Scope
2. Architecture Diagram
3. Component Descriptions
4. Data Flow
    4.1 Ingestion Flow
    4.2 Query Lifecycle
5. Technology Choices
6. Scalability Considerations
7. Evaluation Criteria Mapping

1. System Overview

1.1 Problem Definition

Traditional customer support systems rely on static FAQ databases or rule-based chatbots that fail when queries are nuanced, contextual, or outside a predefined decision tree. As organisations accumulate large knowledge bases in document form, there is a pressing need for an intelligent system that can dynamically retrieve relevant information, synthesise a coherent answer using a large language model, and gracefully escalate to a human agent when confidence is insufficient.

1.2 Scope

- Ingest a PDF knowledge base (Customer Service Handbook) into a persistent vector store.
- Accept natural-language queries via a command-line interface.
- Retrieve the top-3 semantically relevant document chunks using embedding similarity.
- Generate contextual answers using LLaMA 3.3 70B hosted on Groq.
- Route responses through a LangGraph workflow with conditional HITL escalation.
- Hand off to a human agent when the query contains complaint keywords or the LLM signals uncertainty.

2. Architecture Diagram

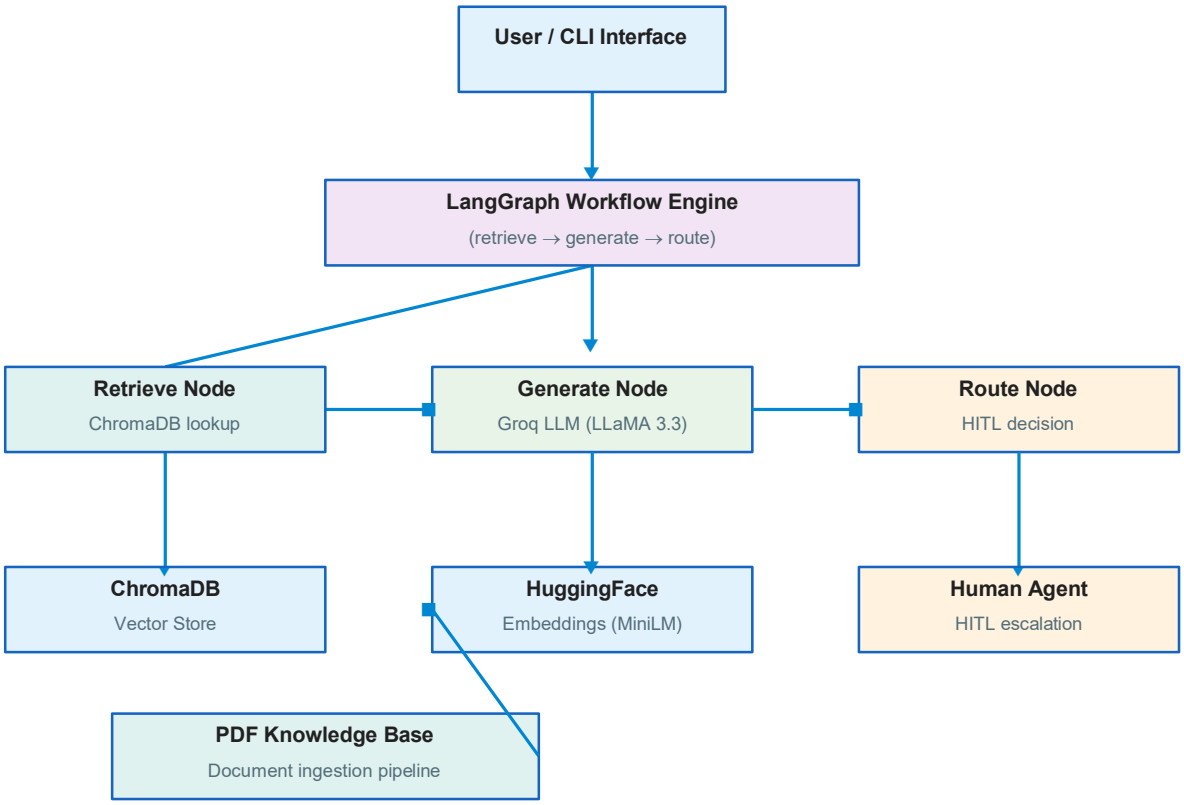

Figure 1 — End-to-end system architecture: PDF ingestion pipeline feeds ChromaDB; LangGraph orchestrates the retrieve → generate → route workflow; HITL escalates to a human agent when needed.

3. Component Descriptions

Document Loader

File: ingestion.py

Uses LangChain's PyPDFLoader to load the source PDF page-by-page, preserving metadata such as page numbers for downstream traceability.

Chunking Module

File: ingestion.py

RecursiveCharacterTextSplitter divides pages into 800-character chunks with a 100-character overlap. The overlap ensures that context spanning chunk boundaries is not lost during retrieval.

Embedding Module

File: ingestion.py / retriever.py

HuggingFace's all-MiniLM-L6-v2 model converts text chunks into 384-dimensional dense vectors. This model was chosen for its balance of speed, size, and semantic accuracy — no API key required.

Vector Store (ChromaDB)

File: ingestion.py / retriever.py

ChromaDB persists embeddings to disk (./chroma_db). On subsequent runs the store is loaded directly, avoiding redundant ingestion. Retrieval uses cosine similarity to return the top-k chunks.

Retrieval Layer

File: retriever.py

Wraps ChromaDB as a LangChain retriever with k=3. Given a user query, it embeds the query and retrieves the three most semantically similar document chunks.

LLM Layer (Groq)

File: graph.py

ChatGroq instantiates the LLaMA 3.3 70B model via Groq's inference API. A structured prompt injects retrieved context and instructs the model to answer only from that context, reducing hallucination.

LangGraph Workflow Engine

File: graph.py

A StateGraph with three nodes — retrieve, generate, route — connected by directed edges. State (a TypedDict) carries query, context, response, escalation flag, and final answer across nodes.

Routing / HITL Module

File: hitl.py

The route node calls should_escalate(), which applies keyword matching (complaint, refund, broken…) and uncertainty detection on the LLM response. On escalation, human_handoff() accepts a live agent response via stdin and injects it as the final answer.

CLI Interface

File: main.py

A while-loop REPL that initialises the graph, accepts user queries, invokes the compiled LangGraph app, and displays either the bot response or the human agent response.

4. Data Flow

4.1 Ingestion Flow (run once)

PDF File → Raw pages loaded by PyPDFLoader

Chunker → Splits into 800-char overlapping segments

Embedding Model → Each chunk converted to 384-dim vector

ChromaDB Vectors + text persisted to ./chroma_db

4.2 Query Lifecycle (every user message)

User Query → Plain-text question entered at CLI

Retrieve Node → Query embedded → cosine search → top-3 chunks returned

Generate Node → Chunks injected into prompt → LLaMA 3.3 generates answer

Route Node → should_escalate() evaluates keywords + uncertainty

Output Bot answer displayed OR human agent prompted for response

5. Technology Choices

Component	Technology	Justification
Document Loader	PyPDFLoader (LangChain)	Native PDF parsing with page-level metadata; integrates directly into LangChain pipeline.
Text Chunking	RecursiveCharacterTextSplitter	Chunk size 800, overlap 100 — balances context richness against retrieval precision.
Embedding Model	all-MiniLM-L6-v2 (HuggingFace)	Lightweight (80MB), no API cost, 384-dim vectors, excellent semantic quality.
Vector Store	ChromaDB (persisted)	Local, zero-config, supports cosine similarity; persists to disk avoiding re-ingestion.
LLM	LLaMA 3.3 70B via Groq	State-of-the-art open model; Groq hardware delivers ~300 tok/s, free tier available.
Workflow Engine	LangGraph 0.2.x	Stateful graph execution; clean node/edge model maps directly to system design.
HITL	Custom Python module	Keyword + uncertainty detection; CLI handoff simulates real escalation pipeline.

6. Scalability Considerations

Large Document Handling

ChromaDB supports millions of vectors. For very large PDFs, chunk size can be reduced and ingestion parallelised using Python's multiprocessing module. A document pre-filtering step can remove boilerplate before chunking.

Increasing Query Load

The current CLI interface is single-threaded. For production, the LangGraph app can be wrapped in a FastAPI endpoint and deployed behind a load balancer. Groq's API supports concurrent requests with rate-limit awareness.

Latency Concerns

Groq's hardware-accelerated inference achieves ~300 tokens/second, making response times typically under 2 seconds. Embedding is local (CPU/GPU), and ChromaDB lookups are sub-100ms for collections under 100,000 chunks.

Multi-Document Support

The ingestion pipeline can be extended to process multiple PDFs by iterating over a directory and maintaining document-source metadata in ChromaDB for source attribution in responses.

HITL at Scale

The current CLI handoff is a demo simulation. In production, escalation would publish to a ticketing system (e.g. Zendesk, Freshdesk) via webhook, decoupling the bot from human response latency.